

Task 1 - Document Intelligence Chatbot

Build a RAG-powered chatbot over a provided PDF, with grounded answers and source citations.

Format Take-home technical assignment	Expected effort Approximately 2-3 hours	AI tools Allowed and encouraged
---	---	---

Please read the full brief before starting. Focus on sound engineering decisions and a clear explanation of trade-offs, not on building a large product.

1. Problem

You are given a PDF containing technical or business information. Build a chatbot that allows a user to ask questions about the contents of the PDF and receive accurate answers grounded in the document.

Important: Use a Retrieval-Augmented Generation (RAG) approach. Do not simply pass the entire PDF to an LLM for every question. Assume the same system may later need to support different PDFs.

2. What you need to build

Document ingestion

- Read and process the PDF.
- Extract useful text and structure where possible.
- Split the document into retrieval-friendly chunks.
- Generate embeddings and store chunks in a searchable representation or vector database.

Question answering

- Accept natural-language questions about the document.
- Retrieve relevant passages for each query.
- Provide the retrieved context to an LLM and generate an answer grounded in that context.
- Support follow-up questions that depend on conversation context.

Citations

- Show the user where the answer came from, for example page number and section heading.

Unknown / unsupported questions

- When the document does not contain enough information, the system should say so rather than inventing an answer.

Example questions

- "What are the eligibility requirements for X?"
- "What is the maximum amount allowed?"
- "How does process A differ from process B?"
- "What does the document say about international applicants?"
- "Who is responsible for this process?"

Example response shape

Answer:
The maximum amount allowed is 50 units.

Source:
Page 47 - "Eligibility Requirements"

Follow-up question example

User: What are the eligibility requirements?
Assistant: ...
User: What about international applicants?
Assistant: ...

3. Engineering requirements

- **Clean structure:** Separate ingestion, retrieval, application/API, and LLM logic where reasonable.
- **Validation:** Validate requests and handle invalid input.
- **Failure handling:** Handle PDF parsing failures, model/API failures, malformed model output, empty retrieval results, and overly long input.
- **Configuration:** Do not hardcode secrets. Model/API configuration should be easy to change.
- **Testing:** Include automated tests for important components and failure paths.
- **Observability:** Where practical, make it possible to understand what was retrieved for a query.

You may choose the technology

Use any programming language, framework, embedding model, vector store, and LLM provider you are comfortable with. A simple API or lightweight UI is sufficient; do not spend significant time on frontend work.

4. RAG evaluation

Create a small evaluation set of approximately 15-20 questions covering different retrieval and reasoning cases.

- Straightforward factual questions
- Questions requiring information from multiple passages
- Follow-up questions that depend on previous context
- Questions whose answers are not present in the document
- Questions where several similar sections could be retrieved

Explain how you would measure

- Retrieval quality - are the right passages being retrieved?
- Answer quality - is the answer correct and supported by the source?
- Citation quality - does the cited source actually support the answer?
- Hallucination / unsupported-answer rate.

You do not need to build a sophisticated evaluation platform. The goal is to demonstrate that you can evaluate an AI system systematically rather than assuming that a working demo means the system is good.

5. Deliverables

- **Source code**
- **README** with setup and run instructions
- **Automated tests**
- **Evaluation dataset** and a short explanation of evaluation methodology
- **Architecture / design explanation**
- **AI_USAGE.md** describing how you used AI coding assistants

In your design explanation, cover

- Your chunking strategy and why you chose it.
- Your retrieval strategy and why you chose it.
- Your embedding / vector-store choice.
- How citations are generated and kept aligned with retrieved content.
- How you handle questions that are not answerable from the document.
- The main limitations of your implementation and what you would improve next.

6. AI usage

AI coding assistants such as Cursor, Claude Code, GitHub Copilot, ChatGPT, or similar tools are allowed and encouraged.

Please include a short **AI_USAGE.md** containing:

- Which AI tools you used.
- What you used them for.
- One example where an AI-generated suggestion was incorrect, unsafe, or suboptimal and you changed or rejected it.

7. Time and scope

Expected effort: approximately 2-3 hours. The goal is not to build a production-ready product. We are evaluating how you approach an AI engineering problem, make trade-offs, test your system, and reason about failure modes.

Optional extensions

- Hybrid search
- Reranking
- Streaming responses
- Semantic caching
- Query rewriting
- Multiple PDF support
- Document-level access control
- Evaluation or observability dashboard

Optional work should not come at the expense of the core requirements.